Early-Stopping Self-Consistency: is the saving still a gain at a matched budget on a small open-weight model?

Roll No. U23AI042 | Syllabus unit: LLMs as reasoning engines (chain-of-thought and self-consistency)

1. Papers read

[1] Primary. Y. Li, P. Yuan, S. Feng, B. Pan, X. Wang, B. Sun, H. Wang, K. Li. Escape Sky-high Cost: Early-stopping Self-Consistency for Multi-step Reasoning. ICLR 2024. arXiv:2401.10480. Code: github.com/Yiwei98/ESC.

[2] Context. X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, D. Zhou. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR 2023.

[3] Context. P. Aggarwal, A. Madaan, Y. Yang, Mausam. Let's Sample Step by Step: Adaptive-Consistency for Efficient Reasoning and Coding with LLMs. EMNLP 2023. arXiv:2305.11860.

2. The primary paper: problem and method

Self-consistency (SC) [2] answers a reasoning question by sampling many chain-of-thought solutions at a non-zero temperature and returning the most frequent final answer. It beats a single greedy answer, but the cost grows in proportion to the number of samples, and that number is fixed in advance for every question, easy or hard. The authors put the price of SC with 64 samples over the MATH benchmark on GPT-4 at about 2,000 dollars.

ESC starts from the point that SC needs only the most frequent answer, not the whole answer distribution. It cuts the budget of L samples into consecutive windows of w samples and, after each window, checks one condition: are all w answers identical? If they are, sampling stops and that answer is returned. If not, the window is kept and the next one is drawn; when no window is ever unanimous, all L samples are voted on exactly as in SC (Algorithm 1). The rule has no threshold to tune, which is the authors' argument against Adaptive-Consistency [3], where a Beta posterior is checked against a 0.95 confidence threshold after every single sample. Two additions support the rule: a z-test argument that an early stop rarely disagrees with full SC (Sec. 2.3), and a control scheme that predicts cost and accuracy for a choice of (w, L) from the first window alone (Sec. 2.4).

The evaluation covers six benchmarks (MATH, GSM8K, CommonsenseQA, StrategyQA, Last Letters, Coin Flip) on GPT-4, GPT-3.5-Turbo and Llama-2 7b, with the few-shot prompts of Wei et al., temperature 0.7, w = 5 and L = 40 (0.5, 8 and 64 for MATH). The headline result is on GSM8K with GPT-4: ESC reaches the same 89.29% as SC with 40 samples while drawing 7.98 samples per question on average (Table 1).

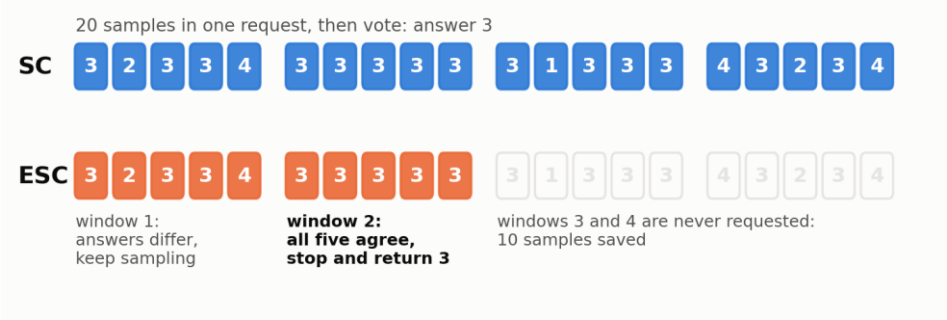


Figure 1. SC and ESC on one question, with the example answers of Figure 1 in [1]. Drawn by me.

3. Research gaps

Gap 1 (evaluation): the matched-budget gain is smaller than the noise nobody reported. Table 1 has a row for SC given the same average number of samples as ESC. On GSM8K, ESC is ahead of it by 0.22 points (GPT-4), 0.85 (GPT-3.5-Turbo) and 0.22 (Llama-2 7b). Sec. 3.2 states that results are averaged over 10 runs and that the variance is omitted for space. On 1,319 questions, 0.22 points is three questions. Without a standard deviation the claim that ESC gets higher accuracy at the same cost cannot be told apart from sampling noise, and this is the claim that matters, since saving samples against SC-40 is trivially possible by running SC-8.

Gap 2 (cost and latency): cost is counted in samples and dollars, never in time. Table 1 counts samples and Table 7 counts tokens and dollars for the two API models; no wall-clock figure appears anywhere. This hides a real price of the method: SC can ask for all L samples in one parallel request, while Algorithm 1 must wait for each window before deciding on the next, up to L/w = 8 sequential requests. For Llama-2 7b on GSM8K, ESC still draws 31.21 of 40 samples, so it saves 22% of the samples but needs about six round trips where SC needs one.

Gap 3 (generality): the saving depends on model strength, and the open-weight evidence is one model that mostly fails. On GSM8K the samples saved fall from 80% (GPT-4) to 63% (GPT-3.5) to 22% (Llama-2 7b); on MATH, Llama-2 uses 62.48 of 64 (Table 1). The paper notes the correlation itself (Sec. 3.2). Llama-2 7b scores 21.75% on GSM8K, so there is no data point for a model of middling accuracy, none for a model small enough to run on a laptop, and the zero-shot check is on GPT-3.5 only (Fig. 4). The 2025 follow-up from the same group (DSC, Findings of NAACL) adds Mistral-7B, and as far as I can find reports neither variance nor latency.

Gap 4 (assumption): the guarantee is about agreeing with SC, not about being right, and it is quoted for a window the experiments mostly do not use. Sec. 2.3 derives z >= sqrt(w) for an early stop on a non-modal answer and quotes a probability of at most 0.002 for w = 8. Five of the six benchmarks use w = 5 (Sec. 3.2), where the same formula gives about 0.013, six times

weaker. Table 5 reports only the overlap between ESC and SC answers. When the model's most likely answer is itself wrong, a unanimous window stops sampling on that wrong answer, and the paper gives no figure for how often a stop is correct.

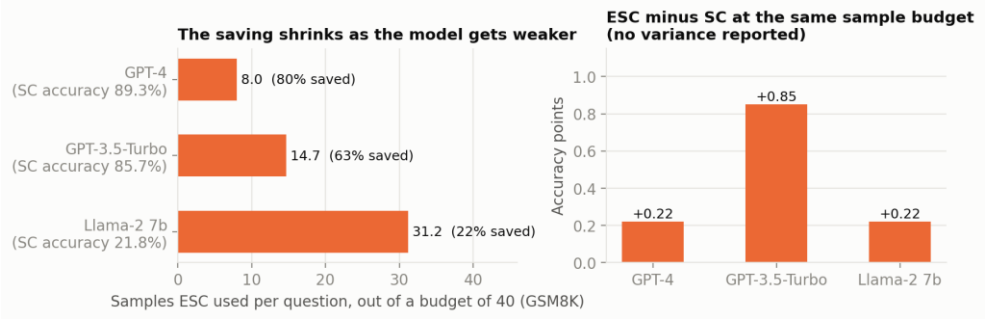


Figure 2. GSM8K numbers from Table 1 of [1], plotted by me: evidence for Gaps 3 (left) and 1 (right).

4. Objective

To evaluate early-stopping self-consistency (window 5, at most 20 samples) for arithmetic word problems on a fixed random subset of 50 questions from the GSM8K test set (1,319 questions), using the open-weight Qwen2.5-1.5B-Instruct model run locally, compared against fixed-size self-consistency at a matched sample budget, measured by exact-match accuracy and by samples, tokens, sequential rounds and wall-clock latency per question, each reported as mean and standard deviation over three seeded runs.

The objective comes from Gap 1: it repeats the paper's own matched-budget comparison with the variance attached. Running it on a 1.5B model places the test in the regime Gap 3 says is missing, and the latency and stop-correctness figures needed for Gaps 2 and 4 fall out of the same traces at no extra cost. The control scheme of Sec. 2.4 is out of scope.

Budget: 20 samples x 50 questions x 3 runs = 3,000 sampled generations, plus 50 greedy calls. At roughly 100 completion tokens each this is about 0.3M completion tokens and 0.25M prompt tokens (a 410-token prompt sent once per window). The model is a 1.1 GB quantised file served by llama.cpp on a laptop CPU at about 25 tokens per second, so the full experiment takes three to four hours and costs nothing. Every other method (SC at any k, Adaptive-Consistency, ablations) is computed from the same stored samples, so the comparison is paired and adds no calls.

Use of language models: an AI assistant (Claude) was used to find and summarise the papers and to help draft this summary; every figure quoted above was checked against the papers.